GENERATIVE AI · FINAL PROJECT DOCUMENTATION

8D Copilot

AI-powered root-cause analysis and 8D report generation for automotive manufacturing quality

STEP 1 Classify ▸ STEP 2 Pareto ▸ STEP 3 Root cause ▸ STEP 4 8D draft ▸ STEP 5 Auditor review ▸ STEP 6 Customer email

Student

NAME	**Feryel Jebali**
MAJOR	**Computer Science Engineering**
COURSE	**Generative Artificial Intelligence**
PROJECT TYPE	**Individual project**
SUBMISSION	**November 2026**

Python · Streamlit · Groq / Gemini API · Prompt engineering · SQLite

01 The problem

Automotive wire-harness plants receive customer complaints every week: swapped pins, terminals backing out of connectors, crimp defects, wrong labels. For each one, a quality engineer must classify it, count the trends for management, find the root cause with 5-Why and Ishikawa, and write an **8D report**, the standard problem-solving document that car makers require from their suppliers.

This takes hours per complaint, and the quality of the report depends on who writes it. **8D Copilot** produces a structured first draft of the whole chain in minutes. The engineer stays in control: every unconfirmed point is marked as a hypothesis.

02 What the application does

8D Copilot is a **Streamlit** web application written in Python. It calls a large language model through an API and processes complaints in a **six-step pipeline**, where each step feeds the next:

Step	What happens	Prompt technique	Temp.
1 Classify	Each complaint gets a defect category, severity, part and summary (JSON)	Few-shot (5 examples), persona, JSON rules	0.0
2 Pareto	Python counts the categories; the AI writes a management summary	Zero-shot, persona	0.3
3 Root cause	Facts, Ishikawa 6M, 5-Why for occurrence and non-detection, verification	Chain of Thought, persona, JSON	0.4
4 8D draft	Full D1–D8 report with action tables; “TBD” for unknown owners and dates	Persona, fixed template	0.5
5 Auditor review	A “strict VDA 6.3 auditor” checks the draft and rewrites it	Iterative refinement	0.2
6 Customer email	Short reply to the customer in a chosen tone	Persona, tone constraint	0.7

Input and export

Paste text, upload CSV, TXT or PDF, or use 18 built-in sample complaints. Export results as CSV and 8D reports as Markdown.

Prompt lab

Compares zero-shot, few-shot and Chain of Thought on the same complaint at several temperatures, side by side.

Prompt log

Shows every API call: the prompt sent, the parameters, the answer, time and tokens.

History and providers

Saved 8D reports in a SQLite database. Works with Groq, Google Gemini and OpenAI through one client.

03 Prompt engineering

All prompts live in `core/prompts.py`, each labelled with its technique, and follow the **3C principle** from the course:

Principle	How it is applied in 8D Copilot
Clarity	The exact task: “category must be exactly one of [list]”, “use exactly these headings”
Context	A role and situation: “senior quality engineer at an IATF 16949 wire-harness supplier”
Constraints	Word limits, allowed values, JSON only, “never invent names, dates or numbers”

Six techniques are used: **zero-shot**, **few-shot**, **Chain of Thought**, **persona**, **structured output** (JSON and templates) and **iterative self-review**.

Experimentation: three prompt versions compared

The same complaint (“two terminals in connector X30 not locked, secondary lock open, intermittent airbag warning”) was sent with three prompt versions at temperature 0.0 and 1.0 (Groq, `openai/gpt-oss-120b`):

Technique	Output	Category	Stable at T = 0.0 vs 1.0?	Tokens
Zero-shot	Free text (Markdown)	Invented: “Mechanical locking failure”	No: severity “High / safety-critical”, then “S1”	222–232
Few-shot + JSON	Valid JSON	Allowed: “Terminal not seated / back-out”	Yes: only the summary wording changed	683–702
Chain of Thought	Steps + JSON at 0.0, JSON only at 1.0	Allowed (same)	Classification yes, but reasoning skipped at 1.0	396–516

Result

Zero-shot was correct in substance but unusable for a Pareto: free text, invented categories, inconsistent severity. **Few-shot** gave stable, parseable answers from the allowed list, so it is used for Step 1, at the cost of about 3x more tokens. Chain of Thought added nothing for this simple task and was not followed consistently at high temperature, so it is used where reasoning matters: the root-cause analysis.

Second refinement

In a test run, the management summary miscounted (“7 of 9 High” instead of 6), even though the correct table was in the prompt. The prompt now says “only quote numbers that appear in the table, do not add or recalculate them”. This confirms the design choice to compute all statistics in Python.

04 Parameter tuning

Each step has its own **temperature**, **max_tokens** and **top_p** (`STEP_PARAMS` in `core/pipeline.py`). Low temperature where consistency matters, higher where natural wording matters. All values can be overridden with sliders in the sidebar.

Step	Temp.	max_tokens	top_p	Reason
Classify	0.0	1500	1.0	The same complaint must always get the same category
Trend summary	0.3	2000	0.9	Factual, with a little freedom in wording
Root cause	0.4	4000	0.9	Some creativity to propose hypotheses
8D draft	0.5	6000	0.9	Long structured document
Auditor review	0.2	7000	0.9	Critical checking, not creativity
Customer email	0.7	2000	0.95	Natural, human tone

Lesson learned during testing

The first version used only 200 tokens for classification. With a “thinking” model, every classification failed: the hidden reasoning tokens count against max_tokens, so the JSON was cut off. The fix: larger budgets, `reasoning_effort = “low”`, and detection of cut-off answers (`finish_reason = “length”`) with a clear message.

05 Error handling

Problem	How the app handles it
Empty, too short or too long input	Warning; limits of 50 complaints and 2,000 characters each
Unreadable CSV or PDF, scanned PDF	Clear message explaining the cause
Missing or wrong API key, wrong model	Stops immediately with a specific message
Rate limit (HTTP 429)	Waits 20 s, then 40 s, and retries (free tiers reset per minute)
Timeout, connection or server error	Retries up to 3 times, then shows a clear message
Invalid JSON from the model	Strips code fences and extracts the JSON; if still invalid, asks the model once to repair it
Category outside the allowed list	Validated in code and replaced by “Other”
One complaint fails in a batch	That row shows its error; the others continue
Answer cut off by max_tokens	Detected with finish_reason; the user is told to raise max_tokens

06 Design decisions

Numbers come from Python

The Pareto is computed with pandas, then given to the model to summarise. The model never produces statistics itself.

No invented facts

An 8D is an official document, so the prompt requires “TBD” for unknown owners and dates and “(hypothesis – to verify)” for unconfirmed causes.

One client, three providers

Groq and Gemini offer OpenAI-compatible endpoints. Groq became the default after Gemini’s free tier hit its daily limit during testing.

Transparency

The Prompt log tab shows and exports every API call, so each answer can be traced back to its prompt and parameters.

07 Structure and how to run

File	Role
`app.py`	Streamlit interface with 5 tabs
`core/llm_client.py`	API calls, retries, error handling, JSON parsing
`core/prompts.py`	All prompts, labelled by technique
`core/pipeline.py`	The six processing steps and their parameters
`core/file_loader.py`	Input validation for text, CSV and PDF
`core/storage.py`	SQLite history of saved reports

Run the app

`pip install -r requirements.txt` then `streamlit run app.py`

Paste a free API key (console.groq.com) in the sidebar, choose “Use sample data” and click **Classify complaints**.

08 Screenshot

Classification results from a real run: 18 sample complaints, no errors.

Results

id	complaint	category	severity	part	summary	error
1	Connector X12 pins 3 and 5 swapped on engine harness, customer EOL electrical test failed,	Wrong wiring / crossed wires	High	Engine harness X12	Pins 3 and 5 swapped on X12, causing EOL test failure and line stop.	
2	Terminal backed out of rear lamp connector when the operator plugged it in at the custome	Terminal not seated / back-out	Medium	Rear lamp connector	Terminal backed out of rear lamp connector during mating at assembly line.	
3	Crimp height too low on ground terminal, wire pulled out with low force during customer au	Crimp defect	Medium	Ground terminal crimp	Crimp height insufficient, wire pulled out easily during pull test.	
4	Missing fuse box clip on cockpit harness, 12 harnesses affected in the last delivery.	Missing or wrong component	Medium	Cockpit harness fuse box clip	Fuse box clip missing on cockpit harness; 12 harnesses affected in recent delivery.	
5	Insulation damaged near the door grommet, copper visible, detected at customer incoming	Damaged insulation / cable	High	Door grommet cable	Insulation damaged near door grommet with exposed copper, detected at customer incomi	
6	Label shows part number revision B, drawing is revision C. Function OK.	Wrong label / marking	Low	Harness label	Label revision B does not match drawing revision C; function unaffected.	
7	Branch to ABS sensor 40 mm too short, operator had to pull the harness to reach the sensor.	Dimension / length out of tolerance	Medium	ABS sensor branch	ABS sensor branch 40 mm too short, requiring harness pulling for installation.	
8	Wrong color wire (blue instead of blue/white) at connector X7 cavity 9, airbag warning light c	Wrong wiring / crossed wires	High	Connector X7 cavity 9	Blue wire used instead of blue/white at X7 cavity 9, causing airbag warning.	
9	Two terminals in connector X30 not locked, secondary lock open, intermittent airbag warnir	Terminal not seated / back-out	High	Connector X30 terminals	Two terminals not locked, secondary lock open, causing intermittent airbag warning.	
10	Box received with 48 harnesses instead of 50, packaging label quantity says 50.	Packaging / logistics	Medium	Harness box	Box contained 48 harnesses but label indicated 50, causing quantity discrepancy.	